

SIMATS ENGINEERING ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO5: Construct interoperable web services by leveraging JAX-RPC, WSDL, SOAP, and XML Schema for seamless data exchange across distributed systems. (L4,L5)

Assessment Tool : Alternative Solution Design

Weightage: 20%

QUESTIONS: 5

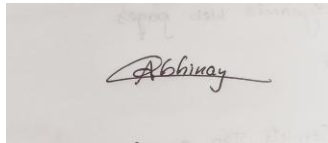
Total Marks: 25

Code of Conduct:

I ____ G. Abhinay (192373062)____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



1. A company needs to develop a web service for a mobile and web application. Analyze both REST and SOAP approaches and design an appropriate solution by selecting one. Justify your choice based on performance, scalability, and security requirements.

Answer:

A) REST vs. SOAP Comparative Analysis:

- Architecture: SOAP is a strict, contract-based protocol (via WSDL) enforcing XML format. REST is an architectural style utilizing standard HTTP methods and multiple payloads (JSON, XML).
- Payload Size: SOAP wraps data in an XML envelope, leading to bloated sizes. REST supports JSON, which is lightweight and fast.
- Security: SOAP has built-in WS-Security (message-level security), while REST relies on transport-level security (HTTPS) and application-level standards (OAuth 2.0 / JWT).

B) Selected Design Solution: REST Web Service (using JSON payload).

C) Justification:

- Performance: Mobile clients operate in variable bandwidth environments; JSON's smaller

payload size drastically reduces network latency and battery consumption compared to parsing heavy SOAP XML blocks.

- Scalability: REST's stateless design allows servers to scale out horizontally easily. HTTP responses can be cached at proxy/gateway layers to serve high read volumes.
- Security: Modern OAuth 2.0 token-based security over HTTPS (TLS 1.3) offers robust, industry-standard access control tailored for web and mobile platforms without the processing overhead of SOAP WS-Security.

2. An online platform expects a large number of users accessing its services simultaneously. Design a scalable service architecture showing how client, server, API, and database interact. Propose an alternative design to improve scalability and justify your approach.

Answer:

A) Baseline Scalable Service Architecture:

- Client Layer: Browser/Mobile client sending asynchronous HTTP requests.
- Load Balancer: Distributes client requests across API servers using a round-robin algorithm.
- API Gateway & App Servers: Serves as reverse proxy, handling routing, rate limiting, and business logic execution.
- Database Layer: Relational database cluster utilizing a Primary-Replica split (reads redirected to replicas, writes to primary).

B) Proposed Alternative Design: Asynchronous Event-Driven Architecture (Message-Queue based).

- Implementation: Introduce an API Gateway that writes write-heavy payloads directly to a Message Queue (e.g., Apache Kafka / RabbitMQ). Background worker microservices consume messages from the queue asynchronously to update the Database. Integrate an in-memory cache (Redis) for fast read requests.

C) Justification:

Decoupling synchronous request flows using message queues prevents database connection exhaustion under sudden traffic spikes. Redis handles read traffic directly, ensuring sub-millisecond responses and leaving relational DB resources free for critical transactional operations.

3. A banking application requires secure communication between client and server. Analyze available communication protocols and select the most suitable protocol. Justify your selection based on security, reliability, and performance.

Answer:

A) Communication Protocol Analysis:

- HTTPS (HTTP/1.1 over TLS 1.2/1.3): Industry-standard secure transport. Stateless, reliable (TCP), but has handshake overhead.
- HTTP/2 (over TLS): Multiplexes requests over a single TCP connection, reducing head-of-line

blocking and connection handshake latency.

- WebSockets (Secure - WSS): Persistent full-duplex TCP channel. Low overhead for real-time notifications, but complex to scale and secure at the gateway.

B) Selected Protocol: gRPC over HTTP/2 with mutual TLS (mTLS) for microservices, and HTTPS (TLS 1.3) for Client-Server.

C) Justification:

- Security: TLS 1.3 guarantees perfect forward secrecy and encrypts all transaction traffic. Mutual TLS verifies identity on both client and server ends.

- Reliability: HTTP/2 keeps connections active, eliminating recurrent TCP handshakes. ProtoBuf enforces strict contracts, preventing structural serialization errors.

- Performance: Multiplexing and binary serialization (Protocol Buffers) compress payloads, boosting transaction speeds and throughput in banking networks.

4. An application needs to integrate with third-party services such as payment gateways or maps. Design an API integration strategy, including request handling, authentication, and response processing. Suggest an alternative approach to improve efficiency or reliability.

Answer:

A) API Integration Strategy Design:

- Request Handling: Set up a dedicated, non-blocking HTTP client connection pool with configurable connection timeouts (e.g. 5 seconds) to prevent external delays from locking thread pools.

- Authentication: Use secure OAuth 2.0 Client Credentials Grant flow. Access tokens are cached locally until expiration to minimize authentication handshakes.

- Response Processing: Handle responses using JSON schema validators. Parse HTTP status codes (2xx success, 4xx client errors, 5xx server failures) and implement retry policies.

B) Proposed Alternative Approach: Webhook-Based Event Driven Integration with Circuit Breaker.

- Implementation: Instead of polling the third-party gateway for transaction states, register a Webhook endpoint. The third party pushes event notifications to our system. Protect endpoints with a Circuit Breaker (e.g., Resilience4j) to temporarily bypass requests if the third party goes offline.

C) Justification:

Webhooks eliminate wasteful polling requests, saving bandwidth and server cycles. Circuit breakers stop cascading failures, maintaining application responsiveness during third-party outages.

5. A web service frequently encounters errors during client requests. Design an error handling and fault tolerance mechanism. Propose alternative strategies to improve system reliability and ensure smooth user experience.

Answer:

A) Baseline Error Handling & Fault Tolerance Mechanism:

- Standardized Error Schema: Return consistent JSON error envelopes (e.g., { "code": "PRODUCT_NOT_FOUND", "message": "Product does not exist." }).
- Global Exception Handler: Use centralized controller advice to map exceptions to standard HTTP codes (400 Bad Request, 401 Unauthorized, 500 Server Error).
- Retries: Apply automatic request retries with exponential backoff for transient 503 Service Unavailable errors.

B) Proposed Alternative Strategies: Circuit Breaker Pattern & Graceful Degradation (Fallback Responses).

- Implementation: Deploy a circuit breaker that monitors downstream service failures. If the failure rate exceeds 50%, the breaker trips (opens), and requests immediately route to a local Fallback Cache or return cached/static datasets instead of throwing server errors.

C) Justification:

This design protects system resources from being wasted on failing backend services. Returning fallback responses ensures that users experience graceful degradation rather than a completely broken application page, protecting the user experience.

**RUBRICS FOR CALCULATION:
Alternative Solution Design**

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly defines problem, objectives, and all requirements accurately	Understands problem with minor gaps	Partial understanding; misses key requirements	Misinterprets or fails to understand problem
Generation of Solutions	25%	Develops multiple innovative and feasible alternatives with clear differentiation	Generates relevant alternatives with minor limitations	Limited or repetitive alternatives	Fails to generate meaningful alternatives
Evaluation of Alternatives	25%	Critically compares alternatives using appropriate criteria and metrics	Compares alternatives with some justification	Basic or superficial comparison	No proper comparison conducted
Solution Selection	20%	Selects best solution with strong justification and decision rationale	Selects suitable solution with moderate justification	Selection is weakly justified	No clear or justified selection
Feasibility	10%	Applies relevant concepts ensuring high feasibility and practicality	Applies concepts with minor issues	Limited feasibility or incorrect application	Solution is impractical or conceptually incorrect